



CAIGE

CENTRO DE ATENDIMENTO
INTERDISCIPLINAR A PESSOA
IDOSA UNIVALE

DOCUMENTAÇÃO TÉCNICA CONSOLIDADA

Guia Técnico do Projeto CAIGE

Arquitetura, tecnologias, instalação, banco de dados, regras de negócio, segurança, permissões e contrato de API da V1.

Versão documental
V1 - Setembro de 2026

Escopo
Projeto, desenvolvimento, implantação e
manutenção

Sumário

Visão consolidada da documentação técnica da V1 do CAIGE.

1. Visão geral do projeto	5
Status documental da V1	5
Estrutura de alto nível	5
Estrutura do repositório	5
2. Perfis e módulos do sistema	6
Perfis da V1	6
Módulos principais	6
3. Arquitetura do CAIGE	7
Frontend	7
Backend	7
Banco de dados	7
Comunicação	7
4. Frontend e Shell Global	8
Arquivos centrais	8
Organização visual	8
5. Tecnologias utilizadas	9
Bibliotecas do frontend	9
Bibliotecas do backend	9
6. Backend e contrato de API	10
Rotas base	10
Convenções HTTP	10
Padrão de erro	10
Foto do paciente	10

7. Banco de dados	11
Scripts principais	11
Grupos de tabelas	11
Relações de destaque	11
8. Integridade e manutenção do banco	12
Prontuário e histórico	12
9. Instalação e execução	13
Pré-requisitos	13
Banco de dados	13
Backend	13
Variáveis de ambiente	13
Inicialização	13
10. PWA e execução no navegador	14
Problemas comuns	14
11. Regras de negócio - visão central	15
Fluxos principais	15
Usuários	15
Pacientes	15
Atividades de atendimento	15
12. Prontuário interdisciplinar	16
13. Frequência e calendário	17
Período letivo	17
Dias de atendimento por atividade	17
14. Bloqueios e cálculo de frequência	18
Bloqueios obrigatórios	18
Percentual de frequência	18

15. Segurança e autenticação	19
Princípio geral	19
Autenticação JWT	19
Identidade autenticada	19
Middlewares	19
Fotos e cache local	19
16. Permissões e escopo por curso	20
Permissões funcionais documentadas	20
Escopo do Professor	20
17. Usuários, arquivamento e credenciais	21
Operações administrativas	21
Arquivamento lógico	21
Ambiente	21
18. Auditoria e movimentações	22
Relação com arquivamento	22
19. Convenções de documentação e manutenção	23
Fontes documentais da V1	23
20. Checklists técnicos	24
Antes de publicar nova rota	24
Após alterar o OpenAPI	24
Boas práticas de instalação e banco	24

1. Visão geral do projeto

O CAIGE é uma aplicação web voltada ao apoio do atendimento interdisciplinar, com foco no cadastro de pacientes, controle de frequência, prontuário, atividades por curso, auditoria e administração de usuários.

Objetivo da V1

Disponibilizar uma versão estável e visualmente padronizada, acompanhada de documentação de usuário, documentação técnica e contrato de API coerentes com o comportamento real do sistema.

Status documental da V1

O pacote revisado de 22/09/2026 apresenta a V1 consolidada com frontend, backend, banco de dados, PWA, revisão geral, documentação técnica e manuais por perfil presentes. Os arquivos Markdown permanecem como fontes editáveis e os PDFs técnicos funcionam como versões visuais de consulta.

Estrutura de alto nível



Estrutura do repositório

```
CAIGE/
├── Backend/
│   ├── database/
│   └── src/
├── Frontend/
│   ├── paginas/
│   └── recursos/
├── docs/
│   ├── usuario/
│   ├── tecnico/
│   └── api/
├── DESIGN_SYSTEM.md
├── REGRAS_PROJETO.md
└── PROMPT_IA.md
```

2. Perfis e módulos do sistema

Perfis da V1

Perfil	Escopo
SUPERVISOR	Acesso administrativo ampliado. Administra usuários, cursos, calendário de frequência, auditoria, arquivados e demais rotinas previstas para o perfil.
PROFESSOR	Perfil operacional vinculado a um curso. Utiliza os módulos liberados e gerencia atividades do próprio curso, sem acesso ao Painel de Gerenciamento.

Módulos principais

- Autenticação
- Painel
- Movimentações
- Pacientes
- Gestão de Prontuários
- Frequência
- Meu Curso (Professor)
- Painel de Gerenciamento (Supervisor)
- Auditoria (Supervisor)
- PWA para instalação em navegador compatível

**Regra de
autorização**

O frontend controla a experiência visual, mas a autorização real permanece no backend. Ocultar botão, aba ou menu não substitui validação no servidor.

3. Arquitetura do CAIGE

A arquitetura da V1 está organizada em três camadas principais: frontend, backend/API e banco de dados MySQL.

FRONTEND HTML + CSS + JavaScript Shell Global	API / BACKEND Node.js + Express JWT + regras de negócio	MYSQL Persistência, integridade e histórico
---	---	---

Frontend

Localizado em Frontend/, contém páginas estáticas, estilos globais e por página, scripts utilitários e módulos de interface, logos/imagens, o Web App Manifest, o Service Worker e os arquivos WebP opcionais de fotos de pacientes em recursos/uploads/pacientes/.

Backend

Localizado em Backend/, concentra inicialização do servidor, rotas, controllers, models, autenticação, autorização, regras de negócio, integração com MySQL, Swagger/OpenAPI e entrega do frontend estático pelo mesmo processo Node.js.

Banco de dados

O banco MySQL mantém a persistência operacional e histórica. Os scripts oficiais da estrutura vigente ficam em Backend/database/.

Comunicação

Frontend e backend comunicam-se por API HTTP/JSON sob o prefixo /api. A documentação interativa é publicada em /api-docs.

4. Frontend e Shell Global

As páginas autenticadas utilizam o Shell Global como base estrutural. Ele centraliza sidebar, cabeçalho, avatar/menu do usuário, navegação principal e comportamento responsivo compartilhado em desktop, tablet e mobile.

Arquivos centrais

- Frontend/recursos/js/shell-global.js
- Frontend/recursos/js/sidebar-menu.js
- Frontend/recursos/js/menu-usuario.js
- Frontend/recursos/js/controle-acesso.js

Organização visual

Arquivo	Responsabilidade
base.css	Tokens, cores, fonte, reset e fundação visual.
layout.css	Sidebar, cabeçalho, largura da página e estrutura compartilhada.
componentes.css	Botões, inputs, cards, grids, tabelas, filtros, badges, modais e paginação.
responsive.css	Comportamento responsivo compartilhado.
paginas/*.css	Somente particularidades reais da página.

Princípio arquitetural

Se duas telas precisam da mesma aparência ou comportamento, a regra deve ser centralizada e não duplicada na página.

5. Tecnologias utilizadas

Camada	Tecnologias
Frontend	HTML5, CSS3, JavaScript Vanilla, IBM Plex Sans, Flatpickr, jsPDF, jsPDF AutoTable, ExcelJS, Web App Manifest, Service Worker e Canvas/WebP.
Backend	Node.js, Express, dotenv, mysql2, jsonwebtoken, bcryptjs, swagger-ui-express e yaml.
Banco	MySQL.
API / documentação	OpenAPI 3 e Swagger UI.

Bibliotecas do frontend

- IBM Plex Sans: tipografia oficial, carregada centralmente por `base.css`, com fallback para Segoe UI, Arial, Helvetica e sans-serif.
- Flatpickr: campos de data, com localização PT-BR por `date-picker-ptbr.js`.
- jsPDF e jsPDF AutoTable: geração e tabelas em PDF.
- ExcelJS: exportação de dados em Excel.
- Web App Manifest + Service Worker: instalação PWA e cache controlado de recursos estáticos.
- Canvas API + WebP: recorte e otimização da foto do paciente no navegador antes do envio.

Bibliotecas do backend

- dotenv: variáveis de ambiente.
- mysql2: conexão MySQL.
- jsonwebtoken: autenticação JWT.
- bcryptjs: hash e verificação de senhas.
- swagger-ui-express e yaml: publicação e leitura do contrato OpenAPI.

6. Backend e contrato de API

A API atende o frontend pelo prefixo `/api`. O contrato oficial é `docs/api/openapi.yaml`, publicado também por Swagger UI em `/api-docs`.

Rotas base

```
/api/autenticacao
/api/usuarios
/api/pacientes
/api/dados-painel
/api/frequencia
/api/arquivo
/api/questionarios
/api/cursos
/api/atividades-atendimento
/api/movimentacoes
/api/periodos-letivos
```

Contrato cruzado

No pacote revisado, os arquivos em ``Backend/src/routes/`` e o OpenAPI possuem o mesmo conjunto de 64 operações HTTP. O único endpoint público é `POST /api/autenticacao/entrar``.

Convenções HTTP

Código	Uso principal
200	Consulta ou atualização bem-sucedida.
201	Criação bem-sucedida.
400	Dados inválidos ou regra de validação.
401	Ausência ou falha de autenticação.
403	Permissão, papel ou escopo negado.
404	Recurso não encontrado.
409	Conflito de regra, quando aplicável.
500	Erro interno.
503	Estrutura do banco desatualizada em pontos específicos.

Padrão de erro

```
{
  "message": "Mensagem legível para o cliente."
}
```

Foto do paciente

- `POST /api/pacientes/{id}/foto`: salva WebP otimizado e exige `pode_editar_paciente`.
- `DELETE /api/pacientes/{id}/foto`: remove o arquivo e exige `pode_editar_paciente`.
- O backend aceita Data URL image/webp e limita o arquivo decodificado a 300 KB.
- O arquivo físico fica em `Frontend/recursos/uploads/pacientes/<id>.webp`; não há BLOB/caminho persistente no MySQL.

7. Banco de dados

O CAIGE utiliza MySQL como banco principal. A estrutura vigente da V1 é mantida pelos scripts oficiais em Backend/database/.

Scripts principais

- CAIGE BANCO LIMPO.sql
- CAIGE BANCO TESTE.sql
- CAIGE_BANCO_BANCA_113.sql

Grupos de tabelas

Domínio	Tabelas
Institucional	cursos, usuarios, permissoes
Pacientes / prontuário	pacientes, perguntas, perguntas_cursos, questionarios, questoes_questionarios, respostas_questionarios
Frequência	atividades_atendimento, frequencia, periodos_letivos, grade_periodo_letivo, excecoes_periodo_letivo
Auditoria	movimentacoes

Relações de destaque

- usuarios.id_curso → cursos.id
- atividades_atendimento.id_curso → cursos.id
- frequencia.id_paciente → pacientes.id
- frequencia.id_atividade → atividades_atendimento.id
- frequencia.id_profissional → usuarios.id
- questionarios.id_curso → cursos.id
- respostas_questionarios.id_profissional → usuarios.id
- movimentacoes.id_usuario → usuarios.id

Foto fora do banco

A foto do paciente não é armazenada no schema MySQL. O backend identifica o arquivo WebP pelo ID do paciente e devolve a URL na API quando disponível.

8. Integridade e manutenção do banco

- `usuarios.email` é UNIQUE e não possui índice simples duplicado.
- `periodos_letivos.semestre` aceita somente 1 ou 2.
- `periodos_letivos.data_inicio` deve ser menor ou igual a `data_fim`.
- `grade_periodo_letivo.dia_semana` aceita somente valores de 1 a 7.
- Filtros de frequência e movimentações usam intervalos sobre `registrado_em` e `criado_em`, preservando índices temporais.
- `usuarios` não mantém o campo legado `setor`; o vínculo institucional do Professor é definido por `id_curso`.
- A tabela `permissoes` mantém apenas flags consultadas pelo backend; rotas administrativas seguem protegidas por `requiresSupervisor`.

Manutenção estrutural

Alterações futuras devem atualizar os scripts oficiais e, quando uma base existente precisar ser migrada, utilizar migration temporária e versionada. Fazer backup antes de alterar tabelas críticas.

Prontuário e histórico

Perguntas podem ser reaproveitadas entre cursos por `perguntas_cursos`. Respostas preservam o profissional responsável e a estrutura aplicada no momento do registro, mantendo o histórico clínico consistente.

9. Instalação e execução

Pré-requisitos

- Node.js em versão LTS
- npm
- MySQL
- Navegador moderno, como Chrome ou Edge

Banco de dados

- 1 Criar um banco MySQL chamado caige.
- 2 Importar o script base em Backend/database/, preferencialmente CAIGE BANCO LIMPO.sql.

Backend

```
cd Backend
npm install
# Windows PowerShell
Copy-Item .env.example .env
# Linux/macOS
cp .env.example .env
```

Variáveis de ambiente

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=sua_senha
DB_NAME=caige
DB_PORT=3306
PORT=3000
NODE_ENV=development
JWT_SECRET=use_uma_chave_longa_aleatoria_e_exclusiva
JWT_EXPIRES_IN=8h
```

Inicialização

```
npm start
# ou em desenvolvimento
npm run dev
```

A aplicação fica disponível em <http://localhost:3000> e a documentação da API em <http://localhost:3000/api-docs>. O frontend estático é servido pelo próprio backend; não há processo Node separado para o frontend.

10. PWA e execução no navegador

O CAIGE pode ser instalado como aplicativo pelo navegador em desktop e dispositivos móveis.

- Em localhost, o Service Worker funciona normalmente para desenvolvimento.
- Fora do ambiente local, a aplicação precisa de HTTPS para que o Service Worker e a instalação PWA funcionem.
- Manifest oficial: Frontend/manifest.webmanifest, com name e short_name iguais a CAIGE.
- Service Worker oficial: Frontend/service-worker.js.
- Ícones de instalação: Frontend/recursos/images/pwa/.

Privacidade local

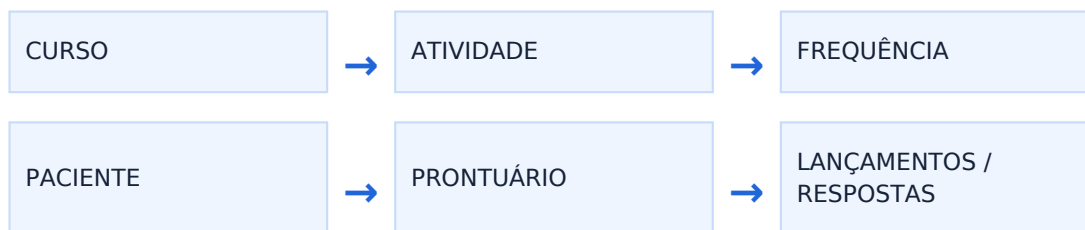
O Service Worker armazena somente recursos estáticos da interface. Respostas de `/api/` e arquivos de `/recursos/uploads/` não entram em cache para evitar persistência local de dados de pacientes.

Problemas comuns

Situação	Verificação
Erro de banco	MySQL ativo; host, porta, usuário e senha no `.env`; banco `caige` existente.
Porta 3000 ocupada	Alterar `PORT` no `.env`.
Login não funciona	Dados de usuário no banco, JWT configurado e backend iniciado sem erro.
Swagger não abre	Confirmar carregamento de `docs/api/openapi.yaml`.

11. Regras de negócio - visão central

Fluxos principais



Usuários

- Criação e edição administrativa são exclusivas do SUPERVISOR.
- Alteração de nome ou senha exige confirmação da senha atual do Supervisor autenticado.
- Nova senha deve possuir no mínimo 6 caracteres.
- Senha em texto puro nunca é armazenada nem registrada em movimentações.

Pacientes

- Podem ser cadastrados, consultados e editados conforme permissões.
- O ciclo operacional utiliza os status ativo, inativo e arquivado.
- A retirada da interface é lógica e preserva o histórico relacional.
- A foto é opcional: JPG/PNG/WebP selecionado no navegador é recortado em Canvas, convertido para WebP e enviado ao backend.
- O backend limita a foto otimizada a 300 KB, salva <id>.webp em Frontend/recursos/uploads/pacientes/ e exige pode_editar_paciente para salvar/remover.
- Sem foto, a interface usa as iniciais do nome como fallback.

Atividades de atendimento

- Cada atividade pertence a um curso.
- PROFESSOR gerencia atividades apenas do próprio curso.
- SUPERVISOR pode administrar atividades de cursos distintos.
- A rota/tabela genérica antiga atividades não integra a V1 final.

12. Prontuário interdisciplinar

- O prontuário possui finalidade clínica.
- Perguntas podem ser vinculadas a um ou vários cursos sem duplicar a pergunta.
- perguntas_cursos mantém os vínculos.
- Questionários pertencem a um curso.
- PROFESSOR cria e altera conteúdo somente no curso autenticado.
- SUPERVISOR possui visão administrativa ampliada.
- A leitura do histórico do paciente é interdisciplinar conforme as rotas de respostas.
- Cada lançamento registra o profissional responsável.
- Quando disponível, a estrutura aplicada no lançamento é preservada por snapshot para não reinterpretar um registro antigo após mudanças no modelo.

Integridade clínica

Alterações posteriores no modelo de prontuário não devem reescrever o significado de registros históricos já lançados.

13. Frequência e calendário

A frequência relaciona paciente, atividade de atendimento e profissional. O momento oficial do registro é `frequencia.registrado_em`. O calendário de frequência é pré-condição funcional para registro e cálculo.

Período letivo

- Ano e semestre 1 ou 2.
- Data inicial e final.
- Status PLANEJADO, ATIVO ou ENCERRADO.
- Períodos não podem se sobrepor.
- Somente um período pode estar ATIVO por vez.
- Período ENCERRADO fica bloqueado para alteração de datas, status, grade e exceções.
- Anos não são hardcoded na interface nem na regra de negócio.

Dias de atendimento por atividade

- Dias seguem padrão ISO de 1 a 7.
- Cada atividade usada em frequência precisa de seus próprios dias de atendimento.
- Não existe fallback dos dias gerais do curso para autorizar presença ou calcular agenda.
- Atividade sem dias cadastrados bloqueia registro e relatórios dependentes daquela agenda.
- Atividade fora do dia previsto não aceita presença.

14. Bloqueios e cálculo de frequência

Bloqueios obrigatórios

- Registro de presença exige data dentro de período ATIVO.
- Período PLANEJADO não aceita presença.
- Período ENCERRADO não aceita novos lançamentos.
- A atividade precisa ter o dia atual explicitamente cadastrado.
- Datas sem atendimento bloqueiam presença.
- Chamadas diretas à API não podem contornar essas regras.
- Relatório é bloqueado sem período/calendário aplicável ou quando atividade necessária não possui agenda.

Percentual de frequência

Percentual de frequência =
$$\frac{\text{encontros com presença em datas previstas}}{\text{encontros previstos válidos}} \times 100$$

O relatório também pode retornar `encontrosPrevistos`, `encontrosComPresenca`, `faltas`, `percentualFrequencia` e `calculoFrequenciaDisponivel`. Registros históricos fora do dia previsto permanecem consultáveis para rastreabilidade, mas não contam como comparecimento.

Sem cálculo inventado

Se não existir período aplicável, atividade necessária sem agenda, encontros válidos ou seleção obrigatória de paciente, o sistema deve informar a condição inválida em vez de produzir um percentual artificial.

15. Segurança e autenticação

Princípio geral

O frontend controla a experiência visual. O backend controla a autorização real.

Autenticação JWT

- 1 Ler Authorization: Bearer <token>.
- 2 Validar assinatura e expiração do token.
- 3 Recuperar o usuário no banco pelo e-mail do token.
- 4 Rejeitar a sessão quando o usuário estiver inativo ou oculto/arquivado.
- 5 Montar req.user.

Identidade autenticada

```
req.user.id  
req.user.email  
req.user.name  
req.user.role  
req.user.idCurso
```

req.user é a fonte confiável de identidade e autoria nas rotas autenticadas.

Middlewares

Middleware	Responsabilidade
requireAuth	Autenticação do usuário e montagem da identidade autenticada.
requirePermission(...)	Consulta permissões funcionais conforme o papel autenticado.
requireSupervisor	Bloqueia a operação quando o papel não é SUPERVISOR.

Fotos e cache local

- Salvar/remover foto de paciente exige JWT e pode_editar_paciente.
- O backend deriva o nome do arquivo somente do ID positivo validado.
- O Service Worker exclui /api/ e /recursos/uploads/ do Cache Storage.
- Fora de localhost, a PWA depende de HTTPS.

16. Permissões e escopo por curso

Permissões funcionais documentadas

- `pode_registrar_frequencia`
- `pode_criar_paciente`
- `pode_editar_paciente`
- `pode_visualizar_paciente`
- `pode_visualizar_relatorios`
- `pode_visualizar_questionarios`
- `pode_gerenciar_atividades`

Escopo do Professor

- Não pode consultar, criar, editar ou desativar atividade de outro curso.
- `GET /api/cursos` retorna somente o curso permitido.
- Questionários e perguntas respeitam o curso autenticado nas operações de gestão.
- Registro e consulta de frequência são limitados ao curso do Professor.
- `GET /api/movimentacoes/recentes` filtra por `movimentacoes.id_curso = req.user.idCurso`.
- Eventos recentes de `/api/dados-painel` aplicam o mesmo escopo.
- Movimentações sem `id_curso` são globais e ficam visíveis ao Supervisor, não ao Professor.
- Filtros enviados pelo cliente não podem ampliar o escopo do usuário.

Regra de confiança

O curso do PROFESSOR deve vir da identidade autenticada no backend, e não de parâmetros enviados pelo cliente.

17. Usuários, arquivamento e credenciais

Operações administrativas

As rotas de `/api/usuarios` e `/api/arquivo/...` são administrativas e exigem SUPERVISOR.

- Alteração de nome e redefinição de senha de outro usuário são operações sensíveis.
- O backend exige `senhaSupervisor` com a senha atual do Supervisor autenticado.
- `novaSenha`, quando informada, deve ter no mínimo 6 caracteres e é armazenada com hash `bcryptjs`.
- O backend não confia em flags de confirmação visual enviadas pelo frontend.

Arquivamento lógico

- O usuário deve ser desativado antes de `DELETE /api/usuarios/{id}`.
- O arquivamento define `ativo=false` e `oculto=true`; o registro não é apagado fisicamente.
- A recuperação restaura `oculto=false`, mas mantém `ativo=false`.
- A conta técnica `suportecaige@univale.br` é protegida contra arquivamento.
- Chamadas diretas à API obedecem às mesmas regras da interface.

Ambiente

- Senhas são armazenadas com hash `bcryptjs`.
- O segredo JWT vem de `JWT_SECRET`.
- O `.env` real não deve ser versionado nem distribuído.
- Em produção, usar uma chave JWT longa e exclusiva.

18. Auditoria e movimentações

A estrutura oficial de auditoria da V1 é a tabela `movimentacoes`. Ações relevantes de cadastro e alteração devem utilizar o mecanismo atual de movimentações.

- Movimentações operacionais de curso registram `id_curso` quando o contexto possuir curso definido.
- SUPERVISOR possui visão global das movimentações.
- PROFESSOR visualiza no Painel e em Movimentações somente registros do curso autenticado.
- Registros sem curso são globais e não entram na visão operacional do Professor.
- Auditoria e histórico de ações usam `movimentacoes` como estrutura oficial.

Relação com arquivamento

Pacientes e usuários arquivados podem ser recuperados pelos fluxos administrativos previstos. A retirada das listagens operacionais é lógica e preserva os relacionamentos necessários ao histórico.

19. Convenções de documentação e manutenção

- Interface e documentação em PT-BR.
- Nomes técnicos são preservados quando fazem parte de contratos, bibliotecas, APIs ou semântica da linguagem.
- Navegação autenticada centralizada no Shell Global.
- Autorização real validada no backend.
- Auditoria consolidada em movimentacoes.
- Calendário de frequência administrado pelo Supervisor.
- Mudanças em regras, contratos ou fluxos documentados devem atualizar a documentação no mesmo ciclo da alteração.

Fontes documentais da V1

Documento	Finalidade
README.md	Visão geral, módulos, stack, estrutura e execução rápida.
docs/GUIA_DO_PROJETO.md	Escopo da V1, perfis, módulos e convenções.
docs/tecnico/TECNOLOGIAS_UTILIZADAS.md	Stack e bibliotecas.
docs/tecnico/ARQUITETURA.md	Camadas, Shell Global, backend e API.
docs/tecnico/INSTALACAO_EXECUCAO.md	Pré-requisitos, ambiente, execução e PWA.
docs/tecnico/BANCO_DE_DADOS.md	Schema vigente, relações e integridade.
docs/tecnico/REGRAS_DE_NEGOCIO.md	Regras funcionais da V1.
docs/tecnico/SEGURANCA_PERMISSOES.md	JWT, autorização e escopo.
docs/api/README_API.md	Contrato, rotas base, status HTTP e validação OpenAPI.
docs/tecnico/GUIA_TECNICO_CAIGE.pdf	Versão visual consolidada deste guia técnico.
docs/tecnico/DESIGN_SYSTEM_CAIGE.pdf	Versão visual do Design System oficial.
docs/tecnico/REGRAS_DESENVOLVIMENTO_CAIGE.pdf	Versão visual das regras de desenvolvimento/manutenção.

20. Checklists técnicos

Antes de publicar nova rota

- Exige autenticação?
- Exige permissão específica?
- É exclusiva de Supervisor?
- Possui escopo por curso?
- O cliente consegue forçar outro idCurso?
- A autoria vem de req.user?
- Dados de erro não expõem segredos?
- O OpenAPI foi atualizado?

Após alterar o OpenAPI

- 1 Validar o YAML.
- 2 Confirmar openapi: 3.0.3.
- 3 Confirmar que não existem \$ref quebrados.
- 4 Iniciar o backend.
- 5 Abrir <http://localhost:3000/api-docs>.
- 6 Comparar os endpoints documentados com Backend/src/routes/.

Boas práticas de instalação e banco

- Manter .env fora do versionamento.
- Não incluir node_modules nos pacotes finais.
- Documentar migrations executadas.
- Realizar backup antes de alterar estrutura de banco.
- Manter documentação e scripts alinhados ao schema vigente.

Regra final

Nunca confiar exclusivamente em restrições visuais do frontend para proteger dados ou operações.

Este PDF é uma apresentação visual consolidada da documentação Markdown oficial. Os arquivos .md e docs/api/openapi.yaml permanecem as fontes editáveis e operacionais do projeto.